



Politécnico
Internacional

ESTRUCTURA DE DATOS

**“Cualquier tecnología suficientemente
avanzada es equivalente a la magia..”**

Arthur C. Clarke

AGENDA

- **Conceptos Básicos de Java**
- **Innova Colombia**
- **Conclusiones**



RAMAS DE LA PROGRAMACIÓN



DESARROLLO WEB

La web es una interfaz universal, las aplicaciones que creas funcionan en cualquier dispositivo.



VIDEOJUEGOS

Trabaja creando tu videojuego favorito para cualquier plataforma.



INTELIGENCIA ARTIFICIAL

Diseña modelos para que las computadoras aprendan y tomen decisiones por sí solas (con Machine Learning).



SEGURIDAD INFORMÁTICA

Rompe la seguridad de un sistema o protégelo de ataques de terceros.



REALIDAD VIRTUAL Y AUMENTADA

Sumerge al usuario en entornos virtuales que simulen la realidad.



DESARROLLO MÓVIL

Desarrolla aplicaciones exclusivas para Android o iOS.

Aprende a programar en cualquier lenguaje (primer curso gratis) en:

 ed.team/programacion





¿QUÉ ÁREA DE LA PROGRAMACIÓN ELEGIR?

- Para comenzar en el mundo de la programación **debes aprender a programar desde cero.**
- Luego puedes ir a **cualquier lenguaje de programación**, dependerá mucho de la rama en la que quieras especializarte, **estas son algunas que puedes seguir:**



Prof. Álvaro Felipe 🇪🇸

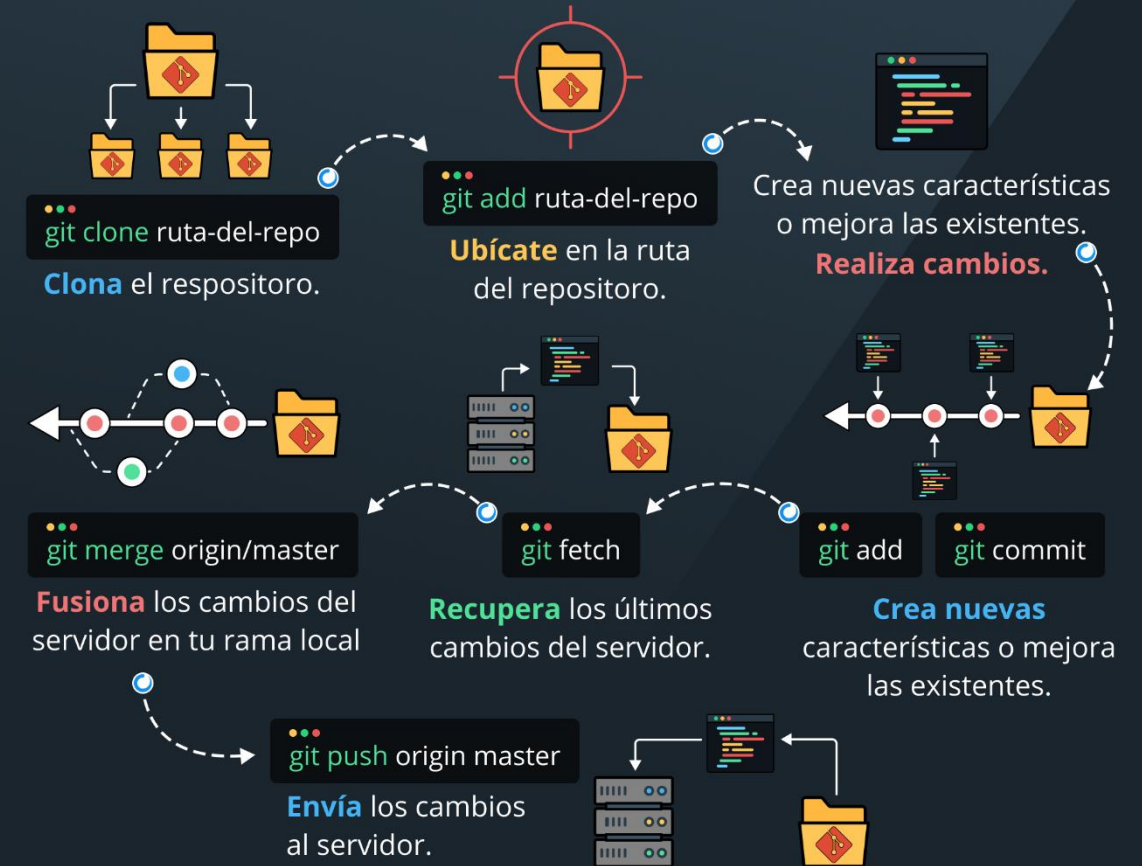
Comienza tu carrera como programador (primer curso gratis) en:

 ed.team/programacion





¿CÓMO CONTRIBUIR EN UN PROYECTO CON GIT?



Aprende a controlar las versiones de tus proyectos:

 ed.team/cursos/git



TÉRMINOS DE git QUE DEBES CONOCER

REPOSITORY/REPO

Base de datos donde se almacena el historial del código.

COMMIT

Registro de uno o varios cambios hechos en el repositorio.

STAGE

Lista de archivos que se usarán para un commit.

FORK

Copia de un repositorio.

BRANCH / RAMA

Entorno o espacio de trabajo independiente en Git.

MASTER

Rama principal de un repositorio.

CHECKOUT

Es la acción de **moverse** entre diferentes ramas.

MERGE

Combina dos o más ramas en una.

Si eres programador es **obligatorio** aprender GIT. Domínalo en:

 ed.team/cursos/git





¿CÓMO FUNCIONA JAVA?



Aprende Java desde cero con EDteam en:

 ed.team/cursos/java





SINTAXIS BÁSICA DE JAVA



Todos los archivos pertenecen a un paquete

Importa los paquetes para el proyecto



Java usa clases para ejecutar el código



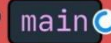
Se debe indicar el tipo de dato.

Modificadores de acceso:

private, public, protected o por defecto ninguno.



El método principal en Java es el método **main**



La palabra reservada **new** crea un objeto del tipo de dato especificado.



Se utilizan ; para cada sentencia

Se usan llaves para identificar el bloque de código

```
package team.ed.course;

import java.lang.*;

public class Person {
    private String name;

    public static void main(String args[]){
        Person friend = new Person();
        friend.name = "Peter";
        System.out.println("Hola " +
            friend.name);
    }
}
```



Aprende Java desde cero hasta nivel Jedi en:

 ed.team/java





*Introducción al Lenguaje De Programación **JAVA***



- *Porque se creo JAVA*
- *Historial de Versiones de JAVA*
- *Características de JAVA*

JAVA

```
void compileFile(final SyntaxNode sn) throws CodeException {
    for (Iterator<SyntaxNode> ite=sn.getChildren().iterator(); ite.hasNext(); ite.next()) {
        final SyntaxNode child = (SyntaxNode) ite.next();
        final Rule rule = child.getRule();
        if (rule.isPackage() || rule.isImport()) {
            final String pack = child.getCharsByRule(RULE_PACKAGE).getTokensChars();
            if (rule.isImport()) {
                //TODO handle static and *
                final SyntaxNode sn = child.getCharsByRule(RULE_IMPORT).getTokensChars();
                final String fullName = ccn.getTokensChars(sn);
                final String[] parts = fullName.split("\\.");
            }
        }
    }
}
```

¿Por qué se creó Java?



Con el inicio de Internet en 1991 el equipo “**Green Team**” teniendo como líder del equipo a **James Gosling** de SUN Microsystems crearon un Lenguaje que inicialmente se llamaba OAK.

Filosofía

“Escribe una vez, ejecuta desde cualquier lado”

JAMES GOSLING

Ingeniero **SUN - GOOGLE**



GOSLING es reconocido como el creador del lenguaje de programación Java. Realizó el diseño original y la implementación del compilador original y la máquina virtual Java, por lo que fue elegido miembro de la Academia Nacional de Ingeniería de Estados Unidos (NAE).

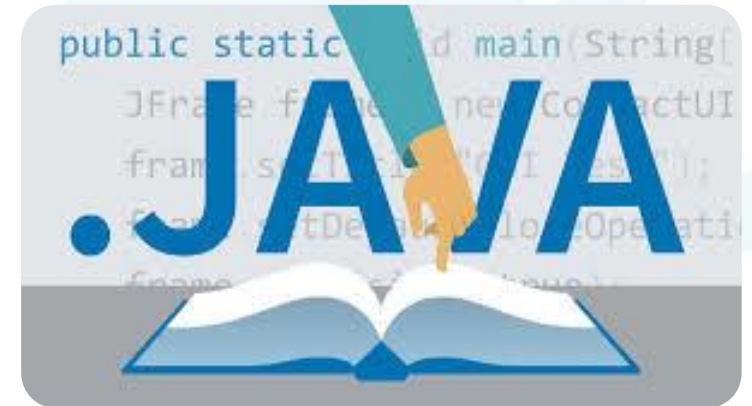
En el 2015 recibió la medalla John von Neumann de la IEEE por sus contribuciones al desarrollo informático

- Junio de 1991 – Se inició el proyecto de lenguaje Java
- JDK 1.0 – enero de 1996
- JDK 1.1 – febrero de 1997
- J2SE 1.2 – diciembre de 1998
- J2SE 1.3 – mayo de 2000
- J2SE 1.4 – febrero de 2002
- J2SE 5.0 – septiembre de 2004
- Java SE 6 – diciembre de 2006
- Java SE 7 – julio de 2011
- Java SE 8 – 18 de marzo de 2014
- Java SE 9 – julio de 2017

CARACTERÍSTICAS DE JAVA

INDEPENDIENTE DE LA PLATAFORMA

Java se creó con la filosofía de “escribe una vez, ejecuta en cualquier lado” (WORA). El código de Java (código Java puro y bibliotecas) que escriba en una plataforma (sistema operativo) se ejecutará en otras plataformas sin modificaciones.



CARACTERÍSTICAS DE JAVA

INDEPENDIENTE DE LA PLATAFORMA

Para ejecutar Java, se utiliza una máquina abstracta llamada [JAVA VIRTUAL MACHINE](#) (JVM). La JVM ejecuta el BYTECODE de Java. Entonces, la CPU ejecuta la JVM.

Dado que todas las JVM funcionan exactamente igual, el mismo código también funciona en otros sistemas operativos, lo que hace que Java sea independiente de la plataforma.



CARACTERÍSTICAS DE JAVA

LENGUAJE ORIENTADO A OBJETOS

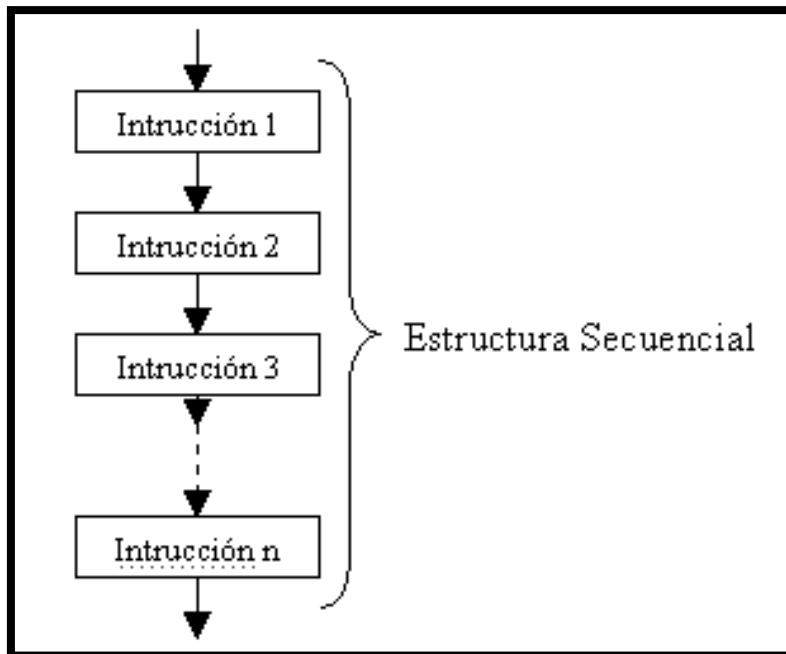
Hay diferentes estilos de programación. El enfoque orientado a objetos es uno de los estilos de programación más popular.

En la programación orientada a objetos, un problema complejo se divide en conjuntos más pequeños mediante la creación de objetos. Esto hace que el código sea reutilizable, tenga beneficios de diseño y haga que el código sea más fácil de mantener.



CARACTERÍSTICAS DE JAVA

PROGRAMACION ESTRUCTURADA



¿QUÉ ES LA PROGRAMACIÓN ORIENTADA A OBJETOS?

Es un paradigma de programación que organiza las funciones en entidades llamadas objetos.

- Los objetos se crean a partir de una plantilla llamada **clase**. Cada objeto es una instancia de su clase.



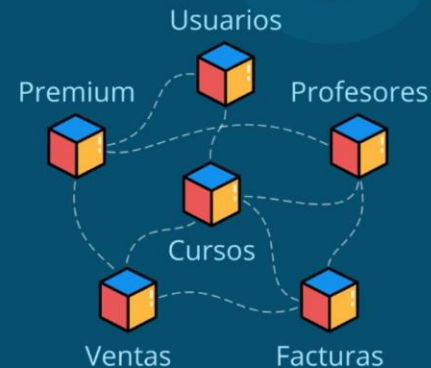
INSTANCIACIÓN

OBJETO



- Los objetos tienen **datos (atributos)** y **funcionalidades (métodos)**.

- En una aplicación los objetos están separados **pero se comunican entre ellos**.



ATRIBUTOS

Nombres
Apellidos
Correo
Contraseña
Premium

MÉTODOS

Editar perfil
Iniciar sesión
Cerrar sesión
Cambiar contraseña
Pasar a premium

Puedes programar con este paradigma **en la mayoría de lenguajes**.



Aprende a crear aplicaciones usando la POO en:

 ed.team/cursos



CARACTERÍSTICAS DE JAVA

SEGURIDAD

- *Proporciona una plataforma segura para desarrollar y ejecutar aplicaciones.*
- *Administración automática de memoria, reduce la corrupción de la memoria y vulnerabilidades.*
- *Proporciona comunicación segura al proteger la integridad y privacidad de los datos transmitidos*



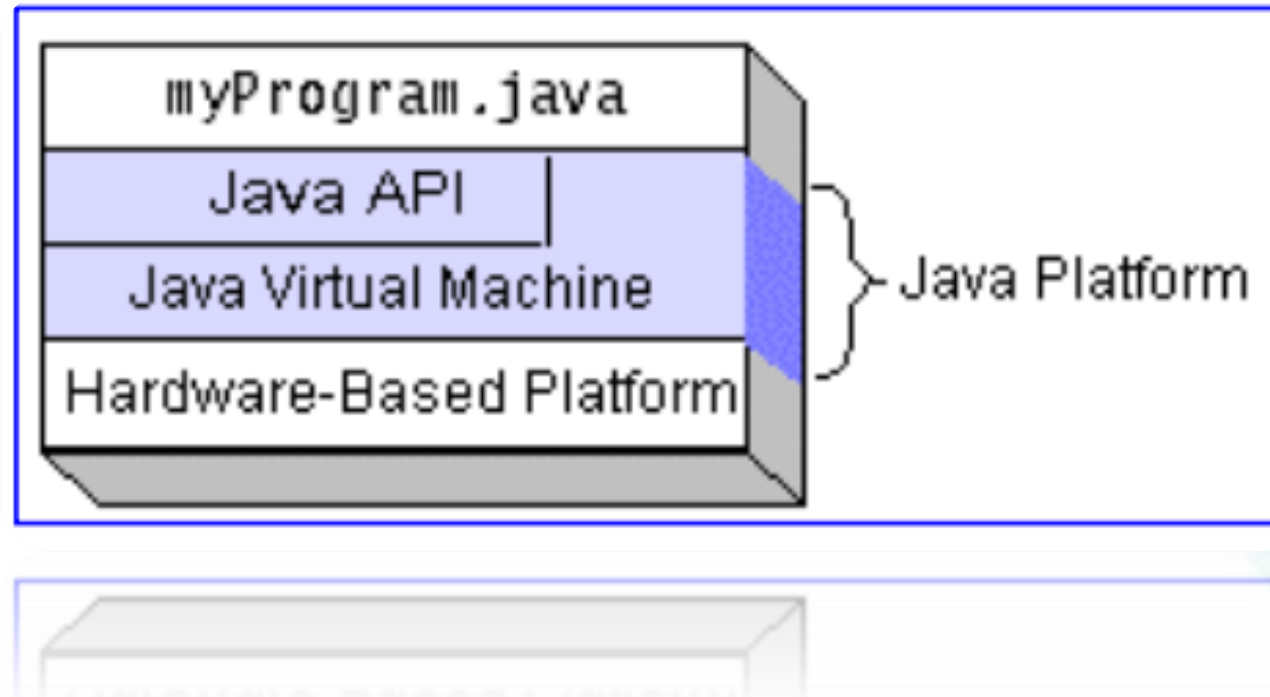
CARACTERÍSTICAS DE JAVA

BIBLIOTECAS

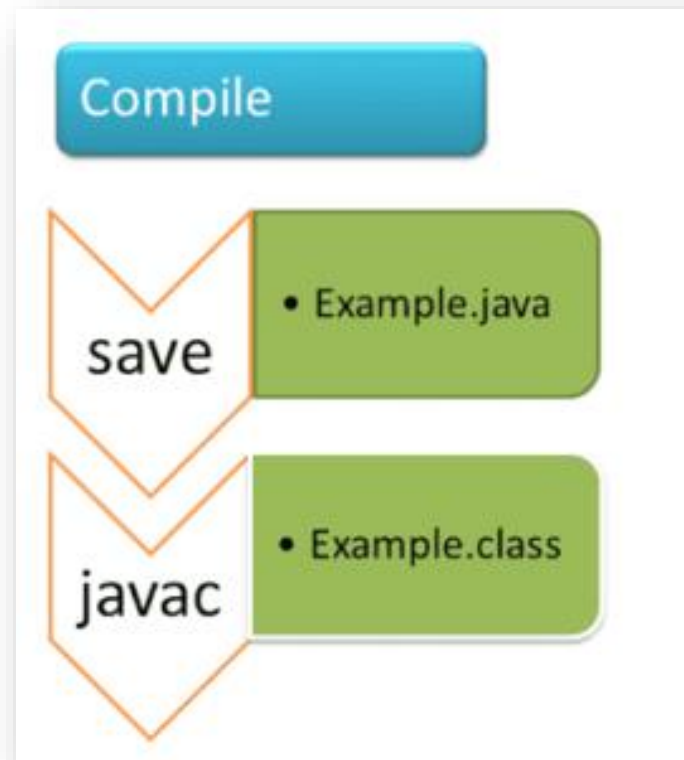
- ***JAVA.LANG*** – Para funciones avanzadas de cadenas, matrices, etc.
- ***JAVA.UTIL*** – Para estructuras de datos, expresiones regulares, funciones de fecha y hora, etc.
- ***JAVA.IO*** – Para archivos E/S, manejo de excepciones, etc.

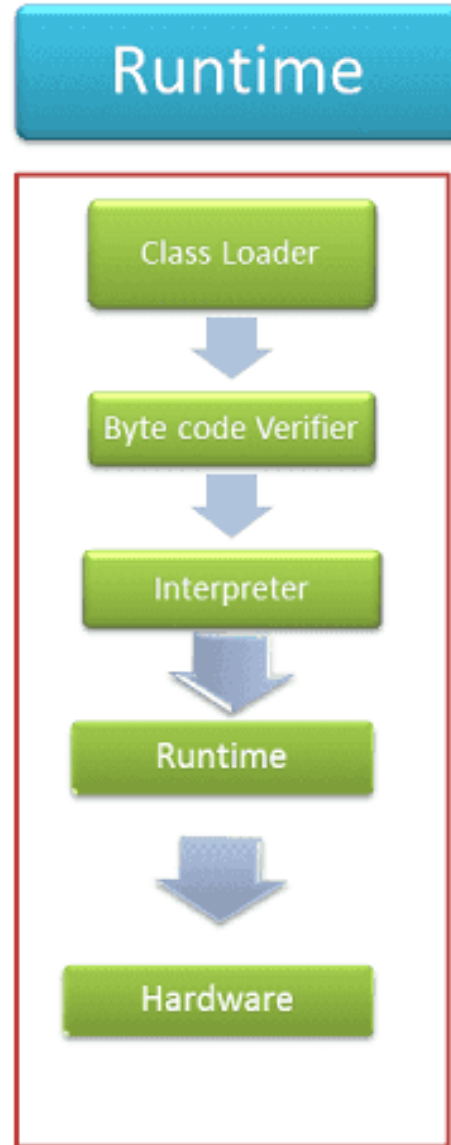


ARQUITECTURA JAVA



JAVA RUNTIME ENVIRONMENT o JRE es un conjunto de utilidades que permite la ejecución de programas JAVA





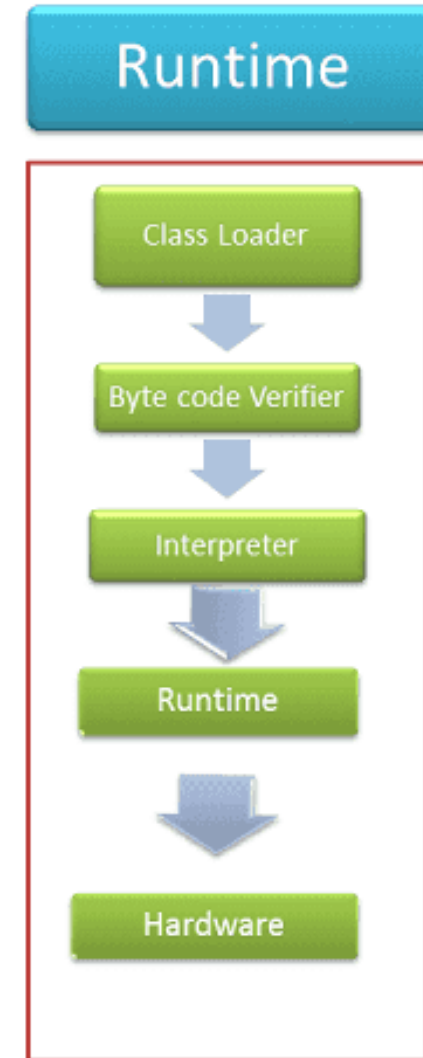
JRE

JAVA RUNTIME ENVIRONMENT

JRE

JAVA RUNTIME ENVIRONMENT

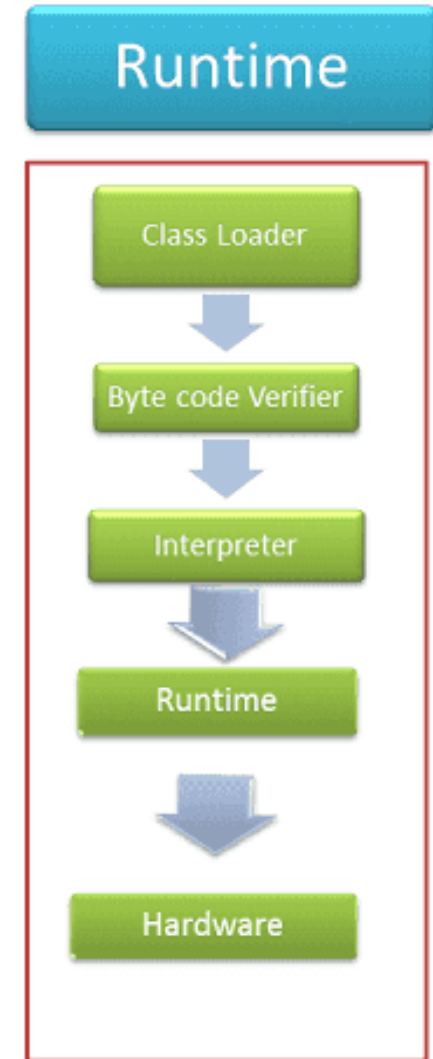
CLASS LOADER: El Class Loader carga todas las clases necesarias para la ejecución de un programa. Proporciona seguridad al separar los espacios de nombres del sistema de archivos local, de los importados a través de la red. Estos archivos se cargan desde un disco duro, una red o desde otras fuentes.



JRE

JAVA RUNTIME ENVIRONMENT

BYTE CODE VERIFIER: La JVM coloca el código en Byte Code Verifier que verifica el formato y busca un código ilegal. El código ilegal, por ejemplo, es un código que infringe los derechos de acceso en los objetos o viola la implementación de los punteros.

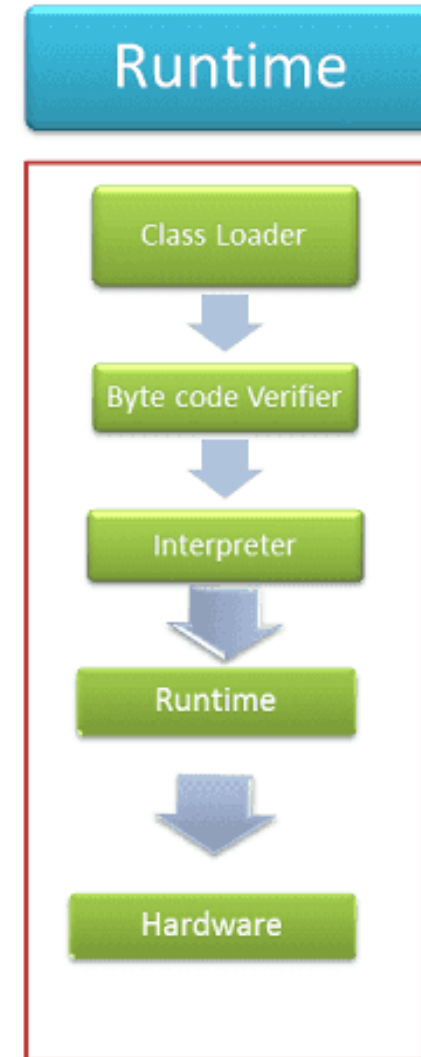


JRE

JAVA RUNTIME ENVIRONMENT

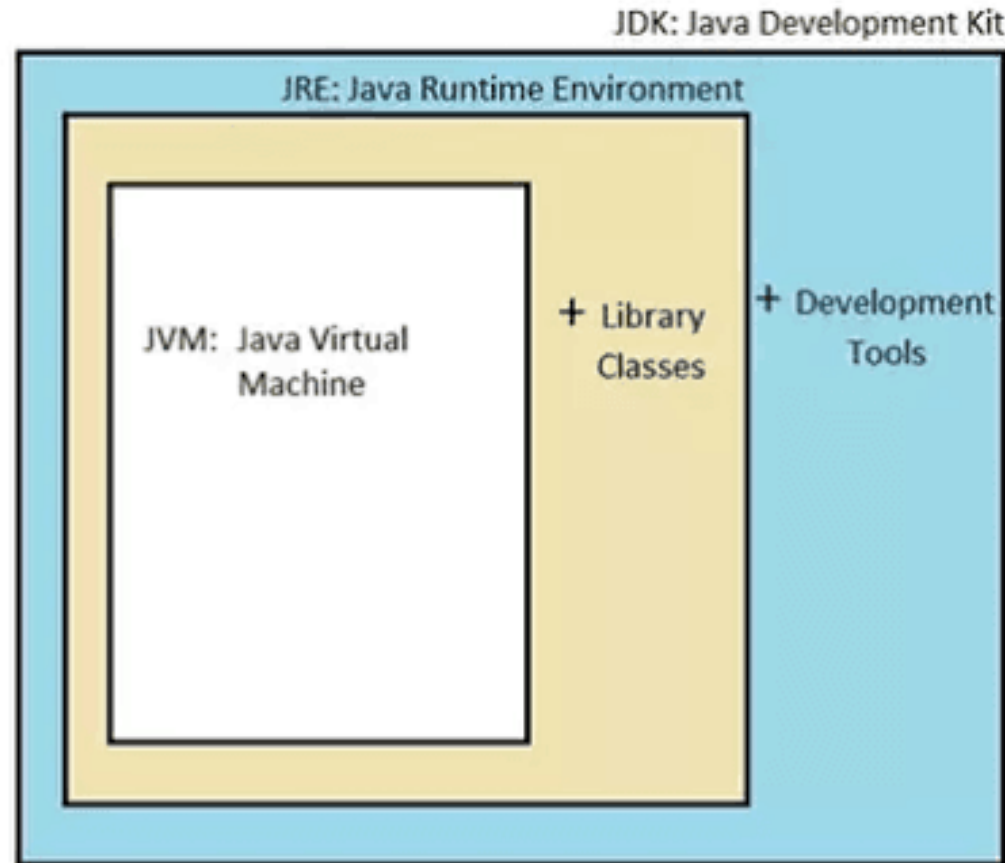
INTREPRETER: *En tiempo de ejecución, el intérprete carga, verifica y ejecuta el Byte Code. El intérprete tiene las siguientes dos funciones:*

- *Ejecute el ByteCode*
- *Hace llamadas apropiadas al hardware subyacente*



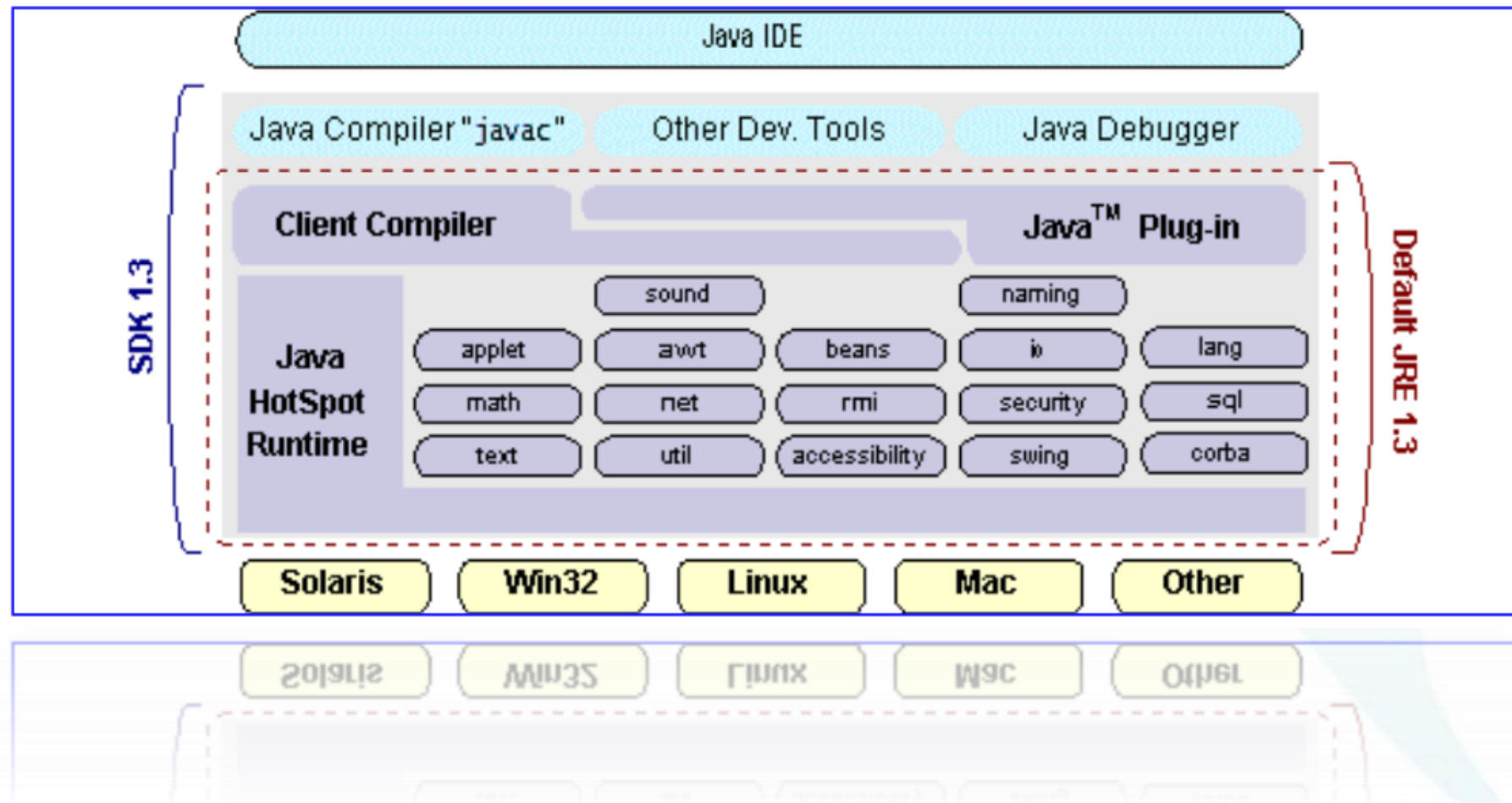
JDK + JVM + JRE

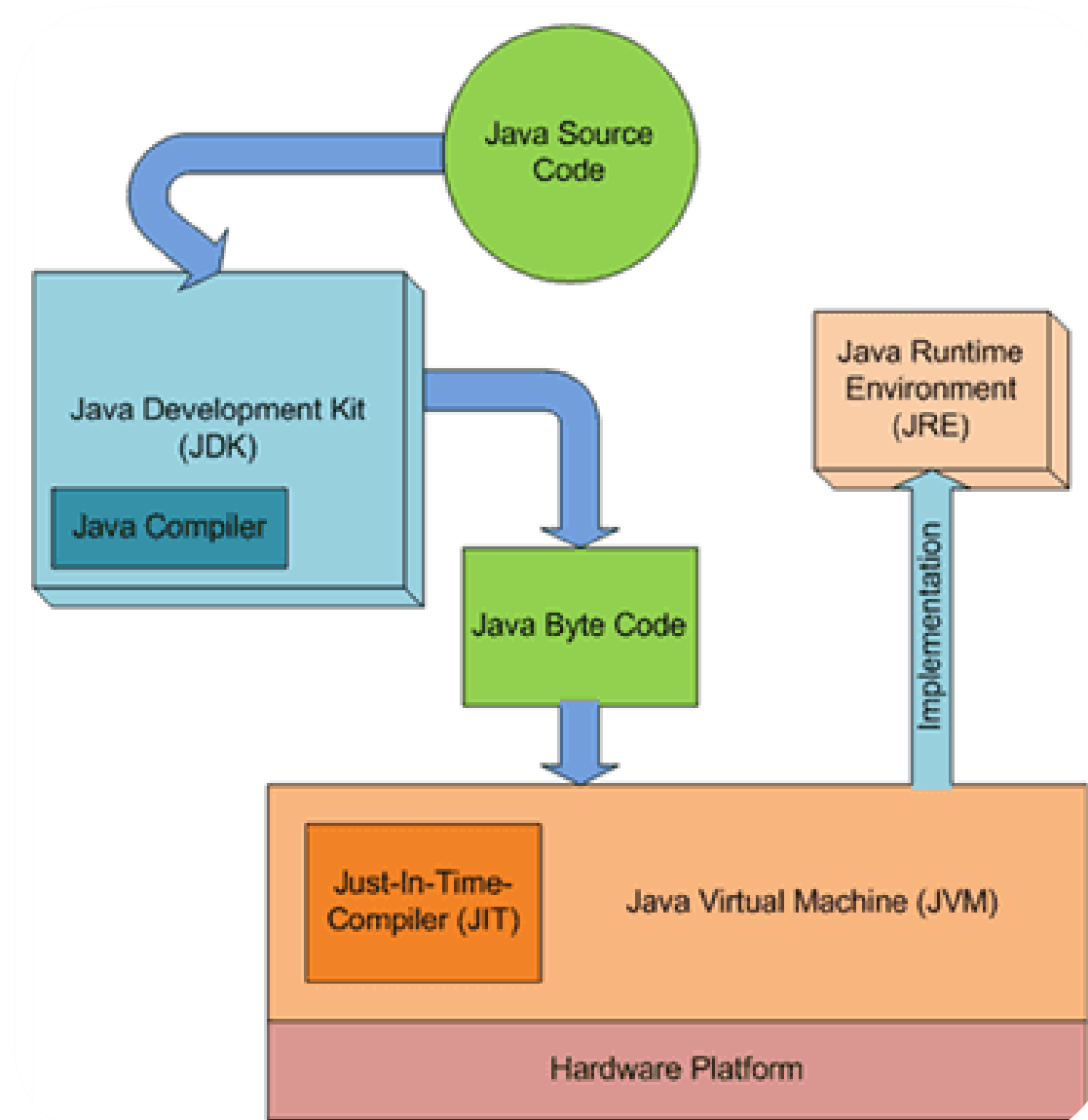
JAVA



JDK = JRE + Development Tools

JRE = JVM + Library Classes







¿CÓMO FUNCIONA JAVA?



Aprende Java desde cero con EDteam en:

 ed.team/cursos/java





SINTAXIS BÁSICA DE JAVA



Todos los archivos pertenecen a un paquete

Importa los paquetes para el proyecto



Java usa clases para ejecutar el código

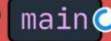


Se debe indicar el tipo de dato.

Modificadores de acceso: private, public, protected o por defecto ninguno.



El método principal en Java es el método **main**



La palabra reservada **new** crea un objeto del tipo de dato especificado.



Se utilizan ; para cada sentencia



Se usan llaves para identificar el bloque de código



```
package team.ed.course;

import java.lang.*;

public class Person {
    private String name;

    public static void main(String args[]){
        Person friend = new Person();
        friend.name = "Peter";
        System.out.println("Hola " +
            friend.name);
    }
}
```



Aprende Java desde cero hasta nivel Jedi en:

 ed.team/java





6 LIBRERÍAS PARA JAVA

Una librería (o biblioteca) es un **conjunto de funciones creada para resolver una necesidad específica.**



1 GOOGLE GUAVA



Mejora del flujo de trabajo

destinado a tareas comunes. (E/S, caché, primitivas, entre otros).

2 RXJAVA



Crea aplicaciones para **reaccionar** a los flujos de datos en tiempo real.

3 MPANDROIDCHART



Crea **gráficos** de líneas, barras, radares y burbujas para Android.

4 FASTJSON



Convierte **objetos** Java en su representación JSON y viceversa.

5 MOCKITO



Librería de código abierto para **simular pruebas unitarias.**



6 JUNIT



Esta es la librería más **usada para pruebas.**

Aprende Java desde cero hasta nivel Jedi en:

 ed.team/cursos/java









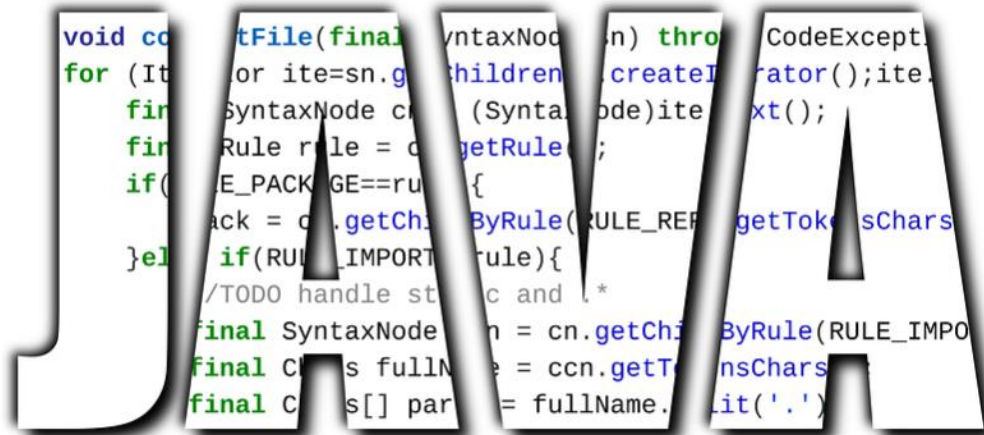


TIPOS DE LENGUAJE



- *Lenguajes Tipo Estático*
- *Lenguajes Tipo Dinámico*

LENGUAJE TIPADO ESTÁTICO



```
void createFile(final SyntaxNode sn) throws CodeException {
    for (Iterator<SyntaxNode> ite=sn.getChildren().iterator(); ite.hasNext(); ite.next()) {
        final SyntaxNode child = (SyntaxNode) ite.next();
        final Rule rule = child.getRule();
        if (rule.isPackage() || rule.isImport()) {
            final String name = child.getChars().toString();
            if (rule.isPackage()) {
                //TODO handle static and final
            } else if (rule.isImport()) {
                final SyntaxNode n = child.getChildByRule(RULE_IMPORT);
                final String fullName = n.getChars().toString();
                final String[] parts = fullName.split("\\.");
            }
        }
    }
}
```



Cada variable y tipo de expresión ya se conoce en tiempo de compilación. Una vez que se declara una variable es de un cierto tipo de datos, no puede contener valores de otros tipos de datos.

Se dice de un lenguaje de programación que usa un tipado estático cuando la comprobación de tipificación se realiza durante la compilación, y no durante la ejecución



LENGUAJE TIPADO DINÁMICO



Estos lenguajes pueden recibir diferentes tipos de datos a lo largo del tiempo. Ejemplo: Ruby, Python

*Un lenguaje de programación **dinámico** es un lenguaje de programación en el que las operaciones realizadas en tiempo de **compilación** pueden realizarse en tiempo de ejecución*



TIPADO ESTÁTICO

JAVA



JAVA está tipado **ESTÁTICAMENTE** y es fuertemente tipado porque en Java, cada tipo de datos (como entero, carácter, hexadecimal, decimal empaquetado, etc.) está predefinido como parte del lenguaje de programación y todas las constantes o variables definidas para un programa dado debe describirse con uno de los tipos de datos.

PRIMITIVOS

&

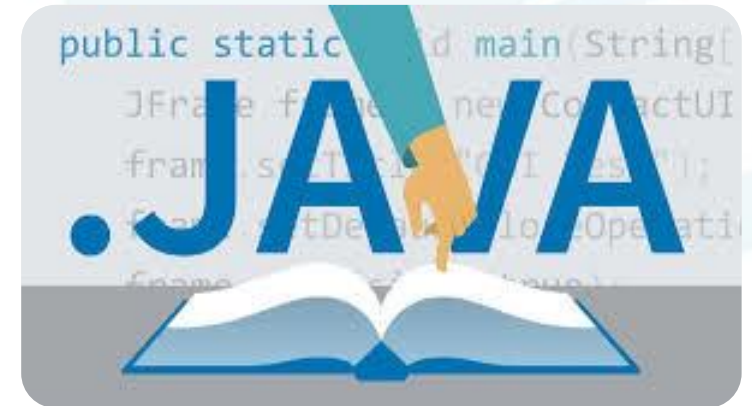
OBJETO



CATEGORIAS DE DATOS

DATOS PRIMITIVOS

Los datos primitivos son solo valores únicos; ellos no tienen capacidades especiales. Java soporta 8 tipos de datos primitivos



CATEGORIAS DE DATOS

DATOS PRIMITIVOS

NUMÉRICOS ENTEROS: Son los tipos *byte*, *short*, *int* y *long*. Los 4 representan números enteros con signo.

CARÁCTER: El tipo *CHAR* representa un carácter codificado en el sistema *unicode*.



CATEGORIAS DE DATOS

DATOS PRIMITIVOS

NUMÉRICO DECIMAL: Los tipos **FLOAT** y **DOUBLE** representan números decimales en coma flotante.

LÓGICOS: El tipo **BOOLEAN** es el tipo de dato lógico; los dos únicos posibles valores que puede representar un dato lógico son **true** y **false**. **true** y **false** son palabras reservadas de Java.



TIPO	DESCRIPCIÓN	DEFAULT	TAMAÑO	EJEMPLOS
boolean	true o false	false	1 bit	true, false
byte	entero complemento de dos	0	8 bits	100, -50
char	carácter unicode	\u0000	16 bits	'a', '\u0041', '\101', '\'
short	entero complemento de dos	0	16 bits	10000,-20000
int	entero complemento de dos	0	32 bits	100000,-2,-1,0,1,2,-200000
long	entero complemento de dos	0	64 bits	-2L,-1L,0L,1L,2L
float	coma flotante IEEE 754	0.0	32 bits	1.23e100f, -1.23e-100f, .3ef, 3.14f
double	coma flotante IEEE 754	0.0	64 bits	1.2345e300d, -1.2345e-300f, 1e1d

TIPO DE DATO BOOLEAN

El tipo de datos booleano representa solo un bit de información: true (verdadero) o false (falso). Los valores de tipo booleano no se convierten implícita o explícitamente (con casts) en ningún otro tipo

True or False:

True

False



Boolean is a
Data Type

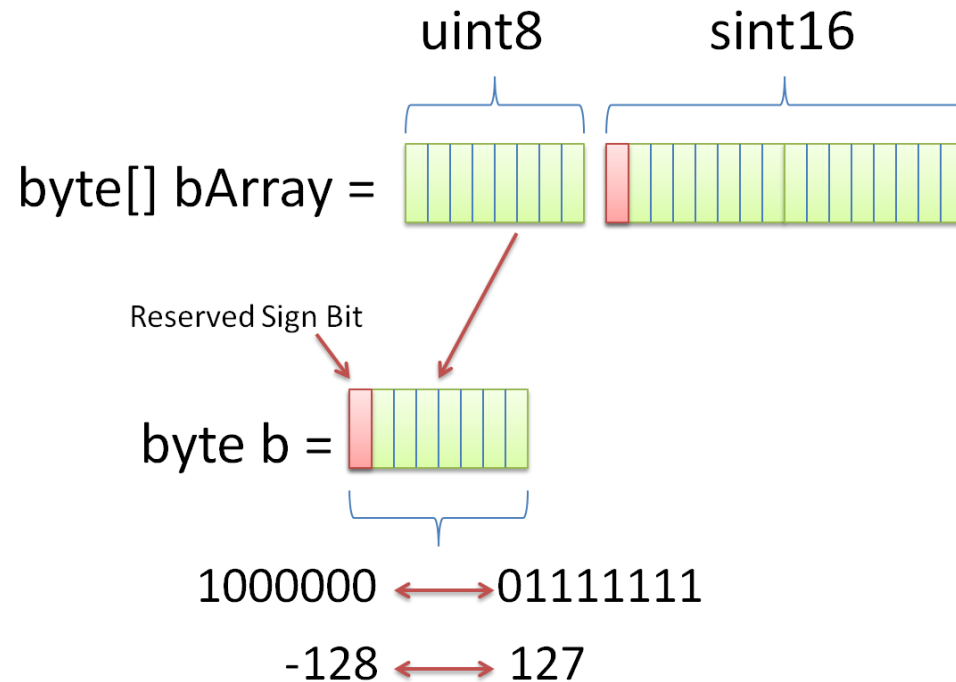
CATEGORIAS DE DATOS

DATOS PRIMITIVOS

TIPO DE DATO BYTE

El tipo de datos byte es un entero de 8 bits de complemento a dos (una forma de representar datos positivos y negativos en binario). El tipo de datos de byte es útil para guardar en memoria grandes arrays.

Tamaño: 8 bits
Valor: -128 a 127



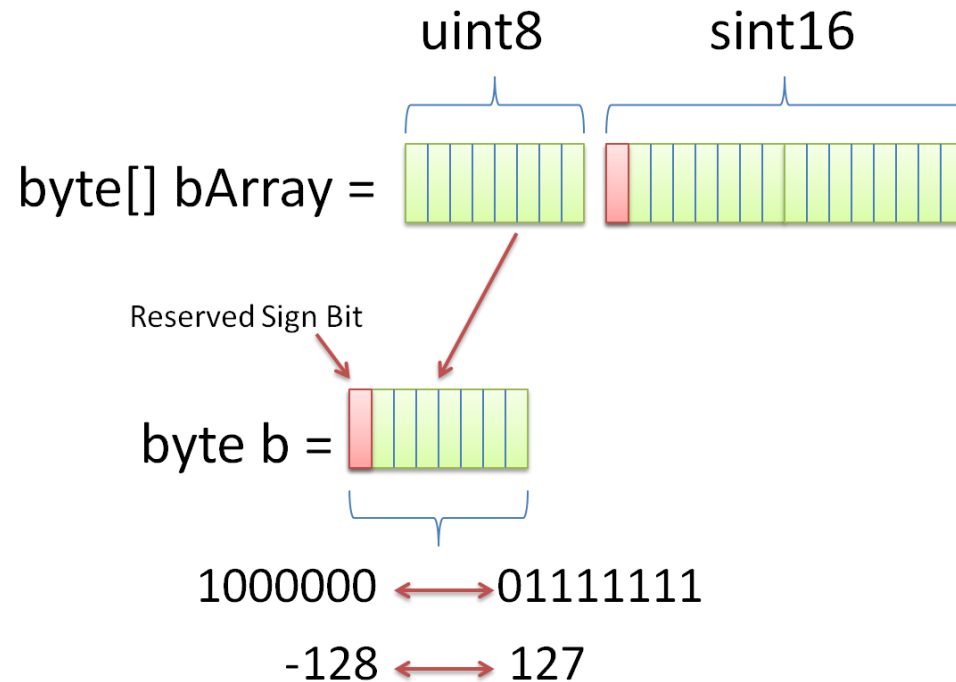
CATEGORIAS DE DATOS

DATOS PRIMITIVOS

TIPO DE DATO BYTE

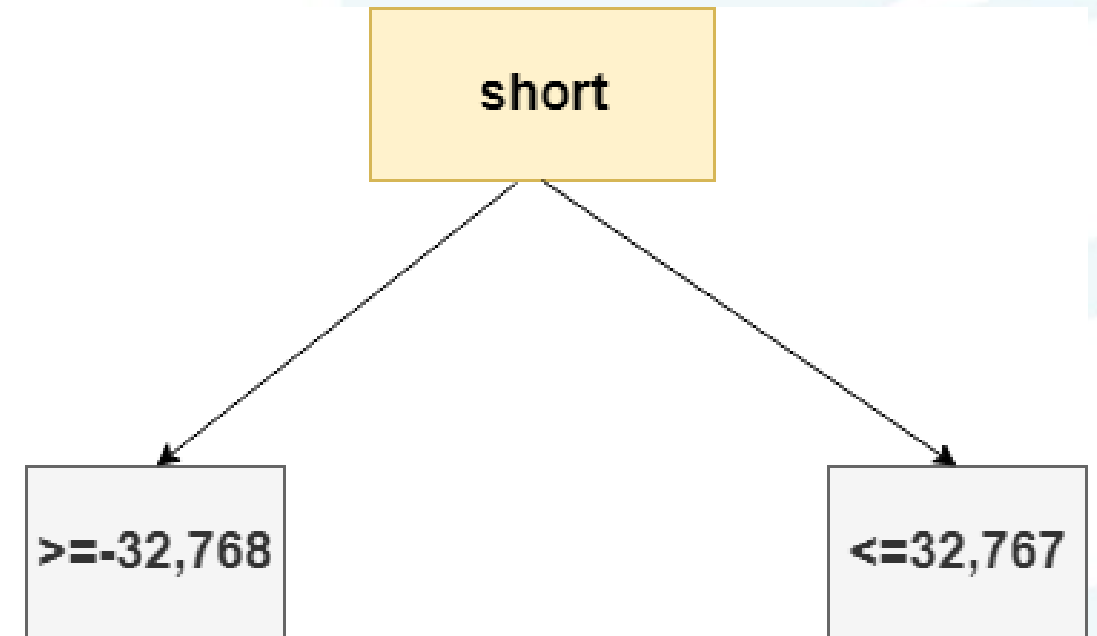
El tipo de datos byte es un entero de 8 bits de complemento a dos (una forma de representar datos positivos y negativos en binario). El tipo de datos de byte es útil para guardar en memoria grandes arrays.

Tamaño: 8 bits
Valor: -128 a 127



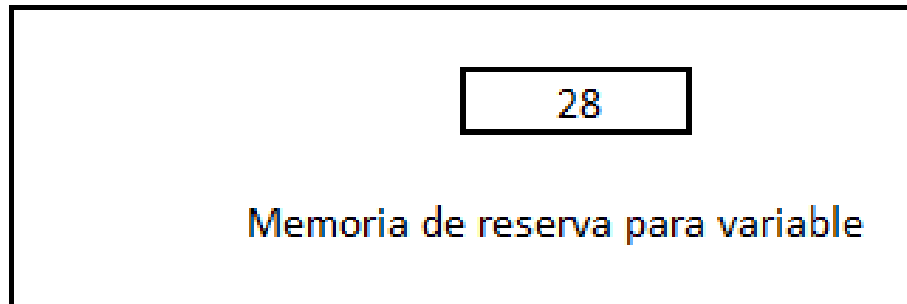
TIPO DE DATO SHORT

*El tipo de datos **SHORT** es un entero de complemento de dos formado por 16 bits. De forma similar al byte, use short para guardar en memoria arrays grandes, en situaciones donde el ahorro de memoria realmente importa.*



`int age = 28;`

tipo de dato nombre_variable valor



RAM

CATEGORIAS DE DATOS

DATOS PRIMITIVOS

TIPO DE DATO INT

Es un entero de complemento de dos formado de 32 bits.

Tamaño: 32 bits

Valor: (-2^{31}) a $(2^{31} - 1)$

*En Java SE 8 y posterior, podemos usar el tipo de datos `int` para representar un entero de 32 bits sin signo, que tiene un valor en el rango $[0, 2^{32} - 1]$. Utilice la clase **INTEGER** para usar el tipo de datos `int` como un entero sin signo.*

CATEGORIAS DE DATOS

DATOS PRIMITIVOS

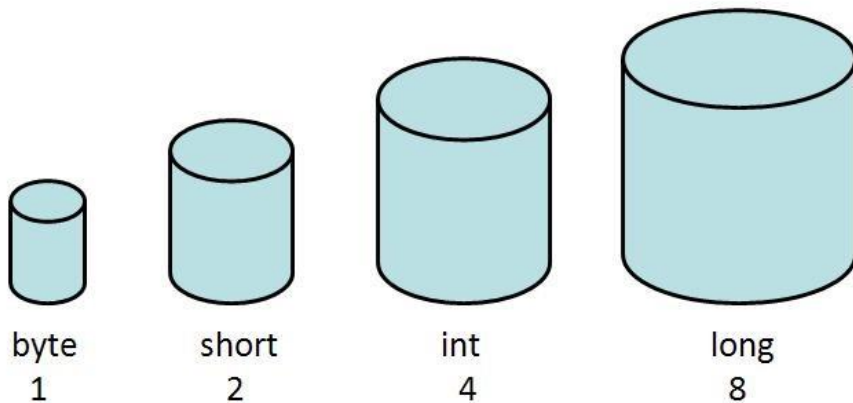
TIPO DE DATO LONG

*El tipo de datos **LONG** es un entero de complemento de dos de 64 bits..*

Tamaño: 64 bit

Valor: (-2^{63}) a $(2^{63}-1)$

*En Java SE 8 y posteriores, puede usar el tipo de datos Long para representar un Long sin signo de 64 bits, que tiene un valor mínimo de 0 y un valor máximo de $2^{64}-1$. La clase Long también contiene métodos como **COMPAREUNSIGNED**, **DIVIDEUNSIGNED**, etc. Para admitir operaciones aritméticas.*



TIPO DE DATO FLOAT - DOUBLE

El tipo de dato float es una coma flotante IEEE 754 de precisión simple de 32 bits. Puede usar float (en lugar double) si necesita guardar en memoria grandes arrays de números de coma flotante.

Tamaño: 32 bits

Sufijo: F/f Ejemplo: 9.8f

El tipo de dato double es una coma flotante IEEE 754 de 64 bits de doble precisión. Para valores decimales, este tipo de datos generalmente es la opción predeterminada.

64bit = double, double precision



32bit = float, single precision



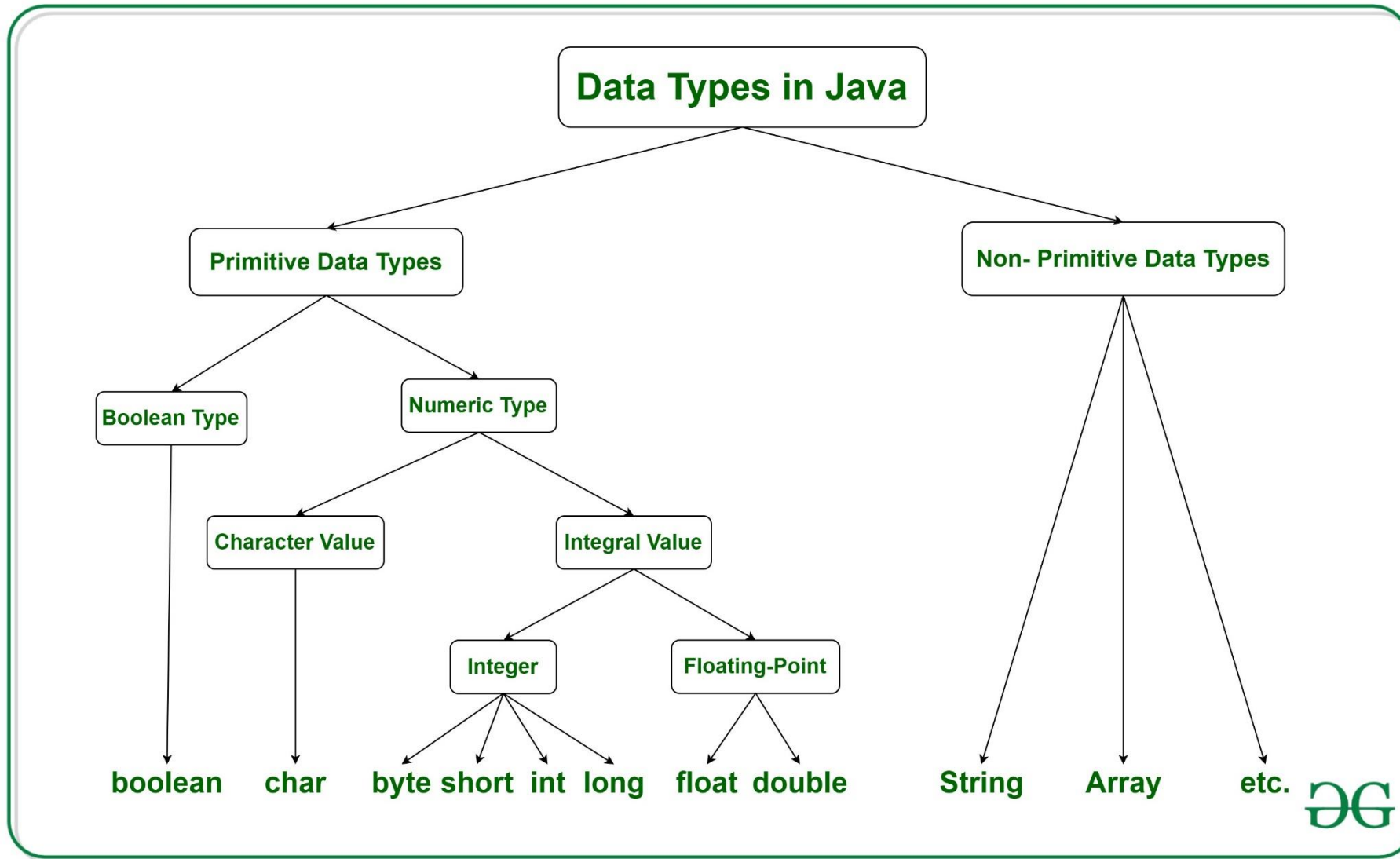
16bit = half, half precision



Signed bit

Exponent

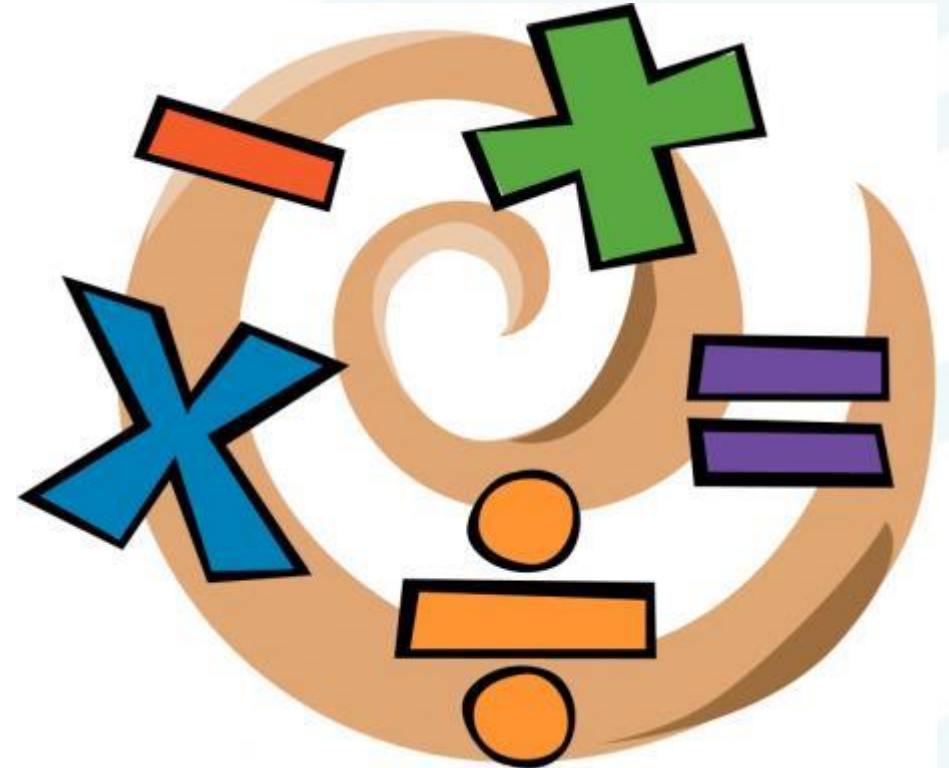
Significand



CLASIFICACION OPERADORES JAVA

Java proporciona muchos tipos de operadores que se pueden usar según la necesidad. Se clasifican según la funcionalidad que brindan. Algunos de los tipos son los siguientes:

- *Operadores aritméticos*
- *Operadores unarios*
- *Operador de asignación*





OPERADORES ARITMÉTICOS

DATOS PRIMITIVOS

JAVA + OPERADORES ARITMÉTICOS

```
// Programa Java para ilustrar
// operadores aritméticos
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c = 0, d = 20, e = 40, f = 30;
        String x = "Thank", y = "You";

        // Operador + y -
        System.out.println("a + b = " + (a + b));
        System.out.println("a - b = " + (a - b));

        // El operador + si se usa con strings
        // concatena las cadenas dadas.
        System.out.println("x + y = " + x + y);

        // Operador * y /
        System.out.println("a * b = " + (a * b));
        System.out.println("a / b = " + (a / b));

        // operador de módulo da el resto
        // de dividir el primer operando con el segundo
        System.out.println("a % b = " + (a % b));

        // si el denominador es 0 en la división
        // System.out.println(a/c);
        // lanzaría una java.lang.ArithmeticException

    }
}
```

Se utilizan para realizar operaciones aritméticas simples en tipos de datos primitivos.

**: Multiplicación*

/: División

?: Módulo

+: Adición

-: Resta



OPERADORES UNARIOS

DATOS PRIMITIVOS

JAVA + OPERADORES UNARIOS

```
// Programa Java para ilustrar
// operadores unarios
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c = 0, d = 20, e = 40, f = 30;
        boolean condition = true;

        // operador de pre-incremento
        // a = a+1 y entonces c = a;
        c = ++a;
        System.out.println("Valor de c (++a) = " + c);

        // operador de post-incremento
        // c=b entonces b=b+1 (b pasa a ser 11)
        c = b++;
        System.out.println("Valor de c (b++) = " + c);

        // operador de pre-decremento
        // d=d-1 entonces c=d
        c = --d;
        System.out.println("Valor de c (--d) = " + c);

        // operador de post-decremento
        // c=e entonces e=e-1 (e pasa a ser 39)
        c = e--;
        System.out.println("Valor de c (e--) = " + c);

        // Operador lógico not
        System.out.println("Valor de !condition = " + !condition);
    }
}
```

Los operadores unarios solo necesitan un operando. Se usan para incrementar, disminuir o negar un valor.

-: Unario menos, utilizado para negar los valores.

+: Unario más, usado para dar valores positivos. Solo se usa cuando se convierte deliberadamente un valor negativo en positivo.

++: Operador de incremento, utilizado para incrementar el valor en 1. Hay dos variedades de operador de incremento.

Pre-Incremento: el valor se incrementa primero y luego se calcula el resultado.

OPERADORES UNARIOS

DATOS PRIMITIVOS

JAVA + OPERADORES UNARIOS

```
// Programa Java para ilustrar
// operadores unarios
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c = 0, d = 20, e = 40, f = 30;
        boolean condition = true;

        // operador de pre-incremento
        // a = a+1 y entonces c = a;
        c = ++a;
        System.out.println("Valor de c (++a) = " + c);

        // operador de post-incremento
        // c=b entonces b=b+1 (b pasa a ser 11)
        c = b++;
        System.out.println("Valor de c (b++) = " + c);

        // operador de pre-decremento
        // d=d-1 entonces c=d
        c = --d;
        System.out.println("Valor de c (--d) = " + c);

        // operador de post-decremento
        // c=e entonces e=e-1 (e pasa a ser 39)
        c = e--;
        System.out.println("Valor de c (e--) = " + c);

        // Operador lógico not
        System.out.println("Valor de !condition = " + !condition);
    }
}
```

Los operadores unarios solo necesitan un operando. Se usan para incrementar, disminuir o negar un valor.

Post-Incremento: el valor se usa por primera vez para calcular el resultado y luego se incrementa.

—: Operador de decremento , usado para incrementar el valor en 1. Hay dos variedades de operador de incremento.

Pre-Decremento: el valor se disminuye primero y luego se calcula el resultado.

Post-Decremento: el valor se usa por primera vez para calcular el resultado y luego se disminuye.

!: Operador lógico “no”, utilizado para invertir un valor booleano.



OPERADORES ASIGNACIÓN

DATOS PRIMITIVOS

JAVA + OPERADORES ASIGNACIÓN

El operador de asignación se usa para asignar un valor a cualquier variable. Tiene una asociación de derecha a izquierda, es decir, el valor dado en el lado derecho del operador se asigna a la variable de la izquierda y, por lo tanto, el valor del lado derecho debe declararse antes de usarlo o debe ser una constante.

El formato general del operador de asignación es, `[java]variable = valor;[/java]`

```
// Programa Java para ilustrar
// Operadores de Asignación
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c, d, e = 10, f = 4, g = 9;

        // operador de asignación simple
        c = b;
        System.out.println("Valor de c = " + c);

        // Esta siguiente declaración arrojaría una exception
        // porque el valor del operando derecho debe ser inicializado
        // antes de la asignación, entonces el programa no
        // compila.
        // c = d;

        // operadores de asignación simples
        a = a + 1;
        b = b - 1;
        e = e * 2;
        f = f / 2;
        System.out.println("a,b,e,f = " + a + ","
                           + b + "," + e + "," + f);

        a = a - 1;
        b = b + 1;
        e = e / 2;
        f = f * 2;

        // operadores de asignación compuestos/cortos
        a += 1;
        b -= 1;
        e *= 2;
        f /= 2;
        System.out.println("a,b,e,f (usando operadores cortos)= " +
                           a + "," + b + "," + e + "," + f);
    }
}
```

```
// Programa Java para ilustrar
// Operadores de Asignación
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c, d, e = 10, f = 4, g = 9;

        // operador de asignación simple
        c = b;
        System.out.println("Valor de c = " + c);

        // Esta siguiente declaración arrojaría una exception
        // porque el valor del operando derecho debe ser inicializado
        // antes de la asignación, entonces el programa no
        // compila.
        // c = d;

        // operadores de asignación simples
        a = a + 1;
        b = b - 1;
        e = e * 2;
        f = f / 2;
        System.out.println("a,b,e,f = " + a + ", "
                           + b + ", " + e + ", " + f);

        a = a - 1;
        b = b + 1;
        e = e / 2;
        f = f * 2;

        // operados de asignación compuestos/cortos
        a += 1;
        b -= 1;
        e *= 2;
        f /= 2;
        System.out.println("a,b,e,f (usando operadores cortos)= " +
                           a + ", " + b + ", " + e + ", " + f);
    }
}
```

OPERADORES ASIGNACIÓN

DATOS PRIMITIVOS

JAVA + OPERADORES ASIGNACIÓN

*En muchos casos, el operador de asignación se puede combinar con otros operadores para construir una versión más corta de la declaración llamada **Declaración Compuesta** (Compound Statement). Por ejemplo, en lugar de $a = a + 5$, podemos escribir $a += 5$.*

$int a = 5;$

$a += 5; // a = a + 5;$



OPERADORES ASIGNACIÓN

DATOS PRIMITIVOS

JAVA + OPERADORES ASIGNACIÓN

```
// Programa Java para ilustrar
// Operadores de Asignación
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c, d, e = 10, f = 4, g = 9;

        // operador de asignación simple
        c = b;
        System.out.println("Valor de c = " + c);

        // Esta siguiente declaración arrojaría una exception
        // porque el valor del operando derecho debe ser inicializado
        // antes de la asignación, entonces el programa no
        // compila.
        // c = d;

        // operadores de asignación simples
        a = a + 1;
        b = b - 1;
        e = e * 2;
        f = f / 2;
        System.out.println("a,b,e,f = " + a + ", "
                           + b + ", " + e + ", " + f);

        a = a - 1;
        b = b + 1;
        e = e / 2;
        f = f * 2;

        // operados de asignación compuestos/cortos
        a += 1;
        b -= 1;
        e *= 2;
        f /= 2;
        System.out.println("a,b,e,f (usando operadores cortos)= " +
                           a + ", " + b + ", " + e + ", " + f);
    }
}
```

+ = , para sumar el operando izquierdo con el operando derecho y luego asignarlo a la variable de la izquierda.

- = , para restar el operando izquierdo con el operando derecho y luego asignarlo a la variable de la izquierda.

- = , para multiplicar el operando izquierdo con el operando derecho y luego asignándolo a la variable de la izquierda.*



OPERADORES ASIGNACIÓN

DATOS PRIMITIVOS

JAVA + OPERADORES ASIGNACIÓN

```
// Programa Java para ilustrar
// Operadores de Asignación
public class operators
{
    public static void main(String[] args)
    {
        int a = 20, b = 10, c, d, e = 10, f = 4, g = 9;

        // operador de asignación simple
        c = b;
        System.out.println("Valor de c = " + c);

        // Esta siguiente declaración arrojaría una exception
        // porque el valor del operando derecho debe ser inicializado
        // antes de la asignación, entonces el programa no
        // compila.
        // c = d;

        // operadores de asignación simples
        a = a + 1;
        b = b - 1;
        e = e * 2;
        f = f / 2;
        System.out.println("a,b,e,f = " + a + ", "
                           + b + ", " + e + ", " + f);

        a = a - 1;
        b = b + 1;
        e = e / 2;
        f = f * 2;

        // operados de asignación compuestos/cortos
        a += 1;
        b -= 1;
        e *= 2;
        f /= 2;
        System.out.println("a,b,e,f (usando operadores cortos)= " +
                           a + ", " + b + ", " + e + ", " + f);
    }
}
```

/ = , para dividir el operando izquierdo con el operando derecho y luego asignarlo a la variable de la izquierda.

^ = , para aumentar la potencia del operando izquierdo al operando derecho y asignarlo a la variable de la izquierda.

% = , para asignar el módulo del operando izquierdo con el operando derecho y luego asignarlo a la variable de la izquierda.



PALABRAS RESERVADAS



En los lenguajes de programación, los identificadores (como su nombre lo indica) se utilizan con fines de identificación. En Java, un identificador puede ser un nombre de clase, un nombre de método o un nombre de variable.

PALABRAS RESERVADAS

```
public class Test
{
    public static void main(String[] args)
    {
        int a = 28;
    }
}
```



IDENTIFICADORES

***TEST:** nombre de clase.*

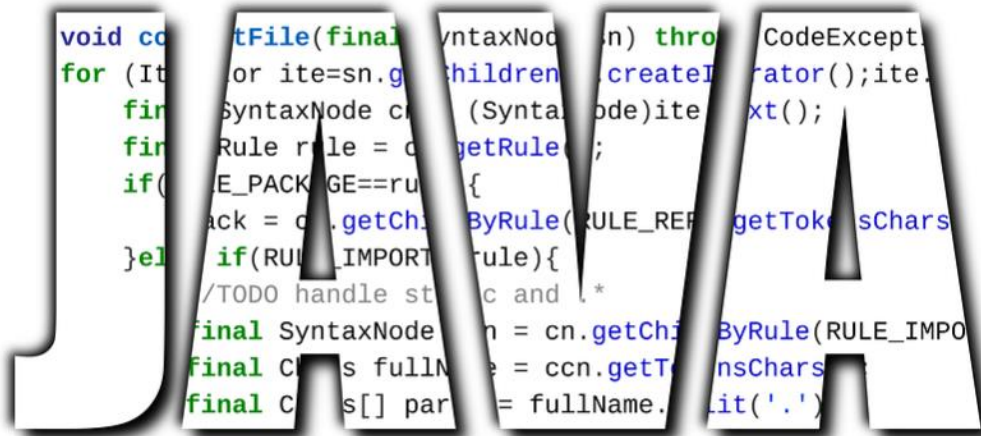
***MAIN:** nombre del método.*

***STRING:** nombre de clase predefinido.*

***ARGS:** nombre de la variable.*

***A:** nombre de la variable.*

REGLAS IDENTIFICADORES JAVA



```
void createFile(final SyntaxNode sn) throws CodeException {
    for (Iterator<SyntaxNode> ite=sn.getChildren().iterator(); ite.hasNext(); ite.next()) {
        final SyntaxNode child = (SyntaxNode) ite.next();
        final Rule rule = child.getRule();
        if (rule.getPackageName() != null) {
            final String pack = child.getCharByRule(RULE_REFERENCE.getTokensChars());
            if (rule.isImport()) {
                //TODO handle static and local
                final SyntaxNode sn = child.getCharByRule(RULE_IMPORT.getTokensChars());
                final String fullName = child.getTokensChars();
                final String[] parts = fullName.split('.');
            }
        }
    }
}
```



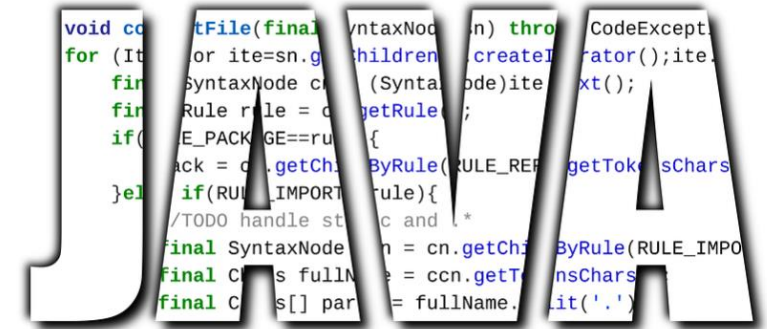
Los únicos caracteres permitidos para los identificadores son todos los caracteres alfanuméricos ([AZ], [az], [0-9]), “\$” (signo de dólar) y ‘_’ (guión bajo). Por ejemplo, “java@” no es un identificador de Java válido ya que contiene “@” – carácter especial.

REGLAS IDENTIFICADORES JAVA



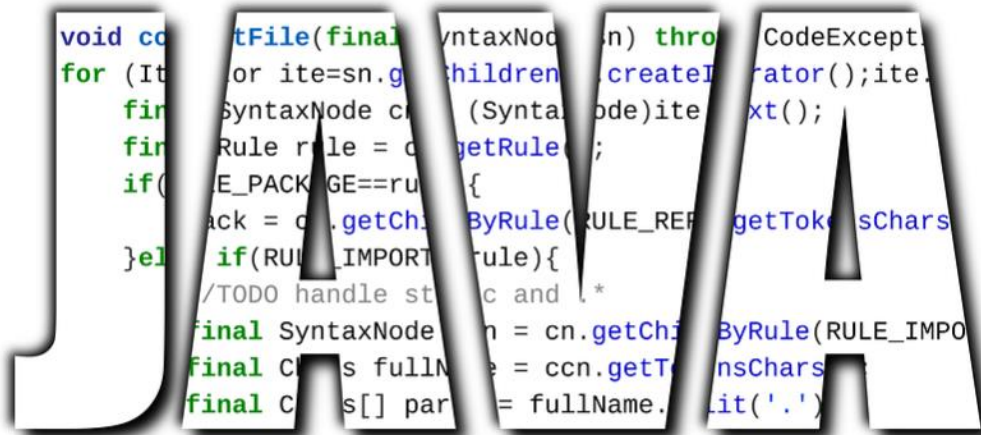
Los identificadores no deben comenzar con dígitos ([0-9]). Por ejemplo, “123java” no es un identificador de Java válido.

Los identificadores de Java distinguen entre mayúsculas y minúsculas.



```
void compileFile(final SyntaxNode sn) throws CodeException {
    for (Iterator<SyntaxNode> ite = sn.getChildren().iterator(); ite.hasNext(); ite.next()) {
        final SyntaxNode child = (SyntaxNode) ite.next();
        Rule rule = child.getRule();
        if (rule == null) {
            // TODO handle static and *
            final SyntaxNode sn = child.getChild(0);
            final String fullName = sn.getTokensChars();
            final String[] parts = fullName.split('.');
        }
    }
}
```

REGLAS IDENTIFICADORES JAVA



```
void createFile(final SyntaxNode sn) throws CodeException {
    for (Iterator<SyntaxNode> ite=sn.getChildren().iterator(); ite.hasNext(); ite.next()) {
        final SyntaxNode child = (SyntaxNode) ite.next();
        final Rule rule = child.getRule();
        if (rule.getPackageName() != null) {
            final String pack = child.getCharsByRule(RULE_REFERENCE.getTokensChars());
        } else if (rule.isImportRule()) {
            //TODO handle static and *
            final SyntaxNode sn = child.getCharsByRule(RULE_IMPORT.getTokensChars());
            final String fullName = child.getTokensChars();
            final String[] parts = fullName.split("\\.");
        }
    }
}
```



No hay límite en la longitud del identificador, pero es aconsejable usar solamente una longitud óptima de 4 a 15 caracteres.

EJEMPLOS IDENTIFICADORES JAVA

BIEN DEFINIDOS

```
MyVariable  
MYVARIABLE  
myvariable  
x  
i  
x1  
i1  
_myvariable  
$myvariable  
sum_of_array  
javadesdecero
```

MAL DEFINIDOS

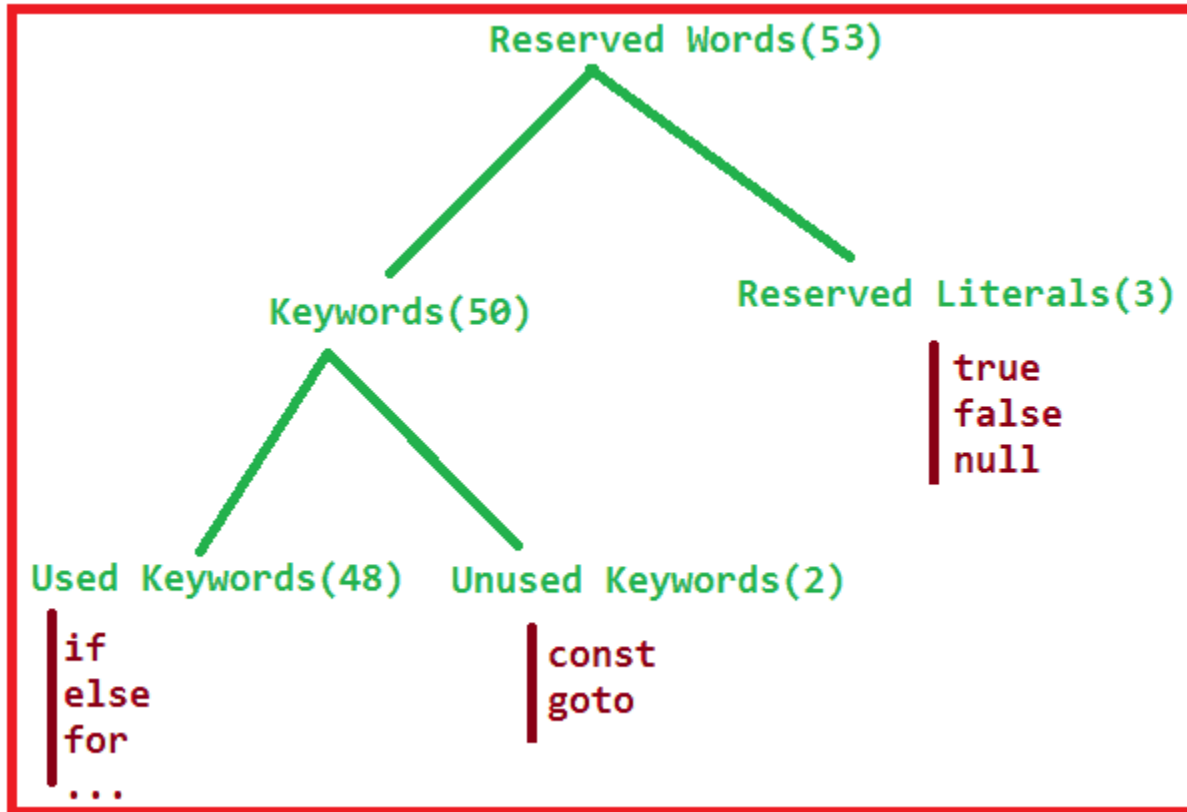
```
My Variable // Contiene un espacio  
123java // Comienza con un dígito  
a+c // El signo más (+) no es un carácter alfanumérico  
variable-2 // El guión (-) no es un carácter alfanumérico  
sum_&_difference // ampersand (&) no es un carácter válido
```



PALABRAS RESERVADAS EN JAVA



Cualquier lenguaje de programación reserva algunas palabras para representar funcionalidades definidas por ese lenguaje. Estas palabras se llaman palabras reservadas. Estas pueden ser categorizadas brevemente en dos partes: palabras clave/keywords (50) y literales (3).



PALABRAS RESERVADAS EN JAVA

Las palabras clave definen funcionalidades y las literales definen un valor.

PALABRAS RESERVADAS EN JAVA

Tabla de Palabras Claves Reservadas en Java

abstract	assert	boolean	break	byte	case	catch	char	class	const	continue
default	do	double	else	enum	extends	false	final	finally	float	for
goto	if	implements	import	instanceof	int	interface	long	native	new	null
package	private	protected	public	return	short	static	strictfp	String	super	switch
synchronized	this	throw	throws	transient	true	try	void	volatile	while	

RETO INNOVA COLOMBIA

¿preguntas?

CONCLUSIONES